

用Agent实现LTM配置梳理与Agent镜像构建

实验背景

为了充分利用大语言模型的自然语言表达能力，提升梳理LTM配置梳理的效率，利用Agent来实现从本地bigip.conf文件中读取所关注的配置信息

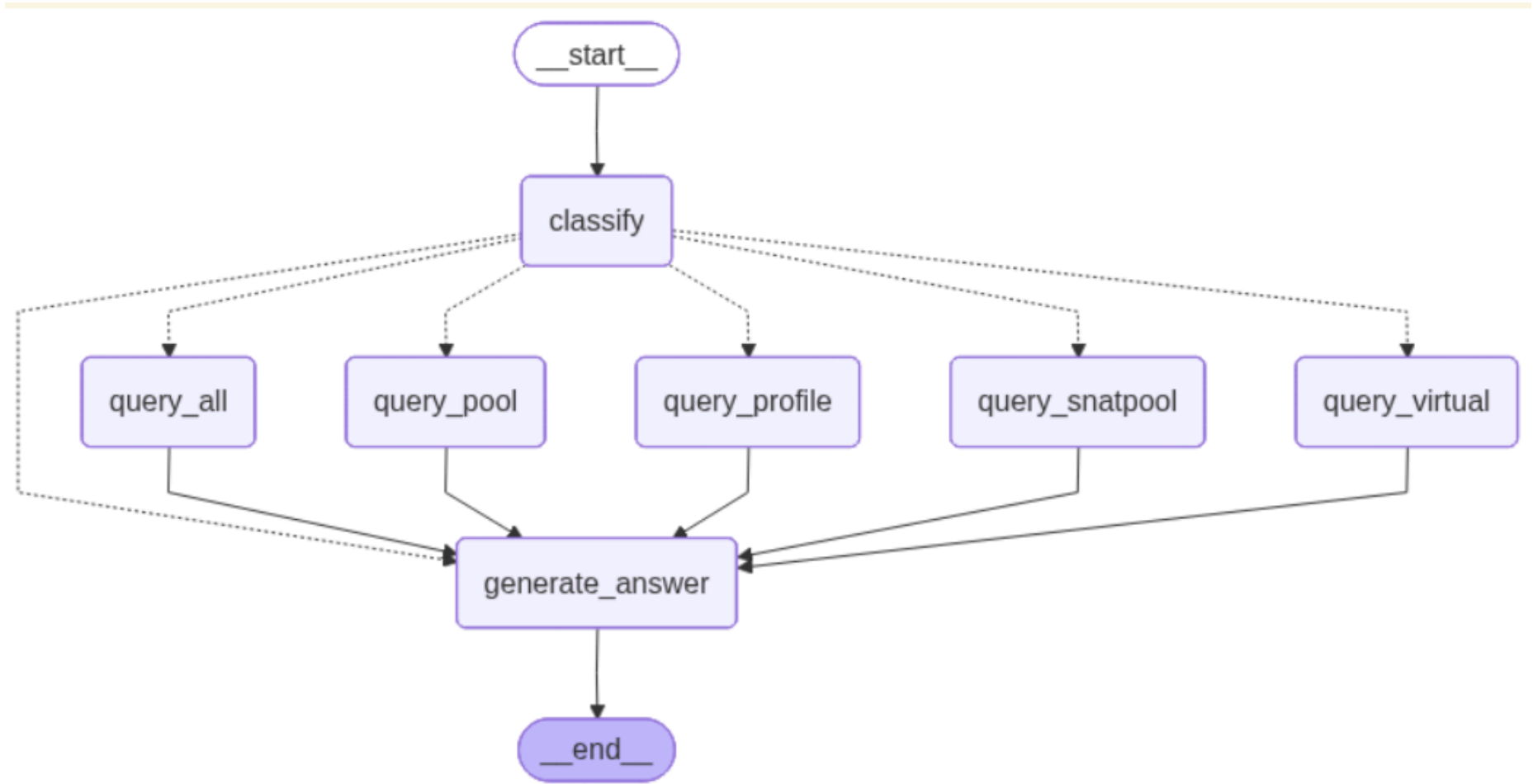
实验效果

如下图所含，输入自然语言，比如 查询所有BIG-IP的配置信息，然后点击查询即可返回 相关配置信息的内容。



实验思路

根据LTM设备的结构化配置信息，分成查询VS,POOL,Profile,SNATPOOL等分项，这样拆分的目的是充分利用Agent的Tools调用能力，实现模块化的代码结构设计思路。



当一个查询请求进来时，如果 **只查询VS信息**，则直接路由到query_virtual节点，并调用绑定的get_virtual tool进行执行和结果**返回**。当查询所有ltm配置时，则在分别调用pool, profile, snatpool, virtual的工具基础上，最后只是封装这些内容在一起，并返回结果。

代码实现

Explorer

Folder

templates

index.html

.env.example

app.py

bigip.conf

Dockerfile

main.py

README.md

requirements.txt

tools.py

README.md (Preview)

README.md

Edit Preview Markdown

BIG-IP 配置查询 Agent

本项目是一个基于 LangGraph + Flask 的 BIG-IP 配置查询系统，支持通过自然语言查询 Virtual Server、Pool、SNAT Pool、Profile 等配置信息。

项目结构

```
1 bigip/
2   ├── app.py           # Flask Web 应用入口
3   ├── main.py          # LangGraph Agent 定义与编译
4   ├── tools.py         # BIG-IP 配置读取工具函数
5   ├── bigip.conf       # BIG-IP 配置文件
6   ├── templates/
7     └── index.html     # Web UI 页面
8   └── README.md        # 本文件
```

上图是整个代码实现的结构说明

```
68 > def query_virtual(state: AgentState) -> dict: ...
72
73 > def query_pool(state: AgentState) -> dict: ...
77
78 > def query_snatpool(state: AgentState) -> dict: ...
82
83 > def query_profile(state: AgentState) -> dict: ...
87
88 > def query_all(state: AgentState) -> dict: ...
96
97 > def generate_answer(state: AgentState) -> dict: ...
121
122 # ===== 条件路由函数 =====
123
124 > def route_query(state: AgentState) -> Literal["query_virtual", "query_pool", "query_snatpool", "query_profile", "query_all", "generate_answer"]: ...
140
141 # ===== 构建流程图 =====
142
143 workflow = StateGraph(AgentState)
144
145 # 添加节点
146 workflow.add_node("classify", classify_query)
147 workflow.add_node("query_virtual", query_virtual)
148 workflow.add_node("query_pool", query_pool)
149 workflow.add_node("query_snatpool", query_snatpool)
150 workflow.add_node("query_profile", query_profile)
151 workflow.add_node("query_all", query_all)
152 workflow.add_node("generate_answer", generate_answer)
```

Agent 的每个Tool， 对应函数为 读取virtual, pool, snatpool,profile 以及上述4个所有配置信息

封装 响应的函数

将上述4个tool,分别加载到Agent的工作节点中

上图是通过Agent实现的思路，重点是每个tool分别加载到对应的Agent 工作节点，实现6个Agent各自独立工作的效果。现实的效果 相当于多个人协同来梳理业务配置，摆脱单人单线程的工作模式。

镜像Agent构建与执行

上述环境只是在本机进行，如果想跨平台进行验证和测试，可以用Docker进行镜像构建，如下是构建步骤。

首先，将源代码文件打开，前提是电脑已经安装好Docker 和Docker Compose

```
root@ubuntuserver:/home/ubuntu/bigip# ll
total 60
drwxr-xr-x 3 root root 4096 Jun 19 00:24 ./
drwxr-x--- 5 ubuntu ubuntu 4096 Jun 19 00:30 ../
-rw-r--r-- 1 root root 1050 Jun 18 15:49 app.py
-rw-r--r-- 1 root root 4905 Jun 18 15:45 bigip.conf
-rw-r--r-- 1 root root 374 Jun 19 00:24 Dockerfile
-rw-r--r-- 1 root root 2291 Jun 19 00:21 .env.example
-rw-r--r-- 1 root root 7010 Jun 19 00:02 main.py
-rw-r--r-- 1 root root 5439 Jun 18 16:07 README.md
-rw-r--r-- 1 root root 115 Jun 19 00:21 requirements.txt
drwxr-xr-x 2 root root 4096 Jun 18 15:43 templates/
-rw-r--r-- 1 root root 7174 Jun 19 00:14 tools.py
```

其次，执行构建命令 docker build -t 你的镜像名 .(当前目录)。

第一次镜像构建，根据网络和电脑性能情况，消耗的时间差异比较大。

```
root@ubuntuserver:/home/ubuntu/bigip# docker build -t agent_bigip:v1 .
[+] Building 2.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 413B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca
=> => resolve docker.io/library/python:3.11-slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca
=> [internal] load build context
=> => transferring context: 308B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:fe9d893527b6a57d22e33295f5b91a8459978765c80e341b65d0c261c4afcdcf
=> => exporting config sha256:19142c687fd6cdc733cc92f03208707808cbb6a7d60d033497ab60b44bc1dcd5
=> => exporting attestation manifest sha256:d1b99f5b72d7bc02c510e08e9cfb2dfc6ba5f99b838c53bbeaf2f2df9e063dfe
=> => exporting manifest list sha256:724108e5f81aa89296e126142dda5a6c8ab4506c56d7cef2fbee2327a3efa655
=> => naming to docker.io/library/agent_bigip:v1
=> => unpacking to docker.io/library/agent_bigip:v1
root@ubuntuserver:/home/ubuntu/bigip# docker images |grep agent
agent_bigip:v1          724108e5f81a          355MB          77.2MB
```

名称自定义

必须是. 表示当前目录

镜像成功后，可以本机看到文件名

最后，镜像构建成功后，执行

`docker run -e ZHIPU_API_KEY=your_api_key申请的KEY_here放这里 -p 5000:5000 bigip-agent:v1`(自己定义的镜像名)

```
root@ubuntuserver:/home/ubuntu/bigip# ip a |grep ens33
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.18.28/24 metric 100 brd 192.168.18.255 scope global dynamic ens33
root@ubuntuserver:/home/ubuntu/bigip# docker run -e ZHIPU_API_KEY=blfa4c7a7c1d9bbfc8j9b9 -p 5000:5000 agent_bigip:v1
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 109-804-003
192.168.18.1 - - [19/Jun/2026 01:06:42] "GET / HTTP/1.1" 200 -
192.168.18.1 - - [19/Jun/2026 01:07:11] "GET / HTTP/1.1" 200 -
192.168.18.1 - - [19/Jun/2026 01:07:23] "GET /favicon.ico HTTP/1.1" 404 -
192.168.18.1 - - [19/Jun/2026 01:07:42] "POST /agent_query HTTP/1.1" 200 -
192.168.18.1 - - [19/Jun/2026 01:07:54] "POST /agent_query HTTP/1.1" 200 -
192.168.18.1 - - [19/Jun/2026 01:08:22] "POST /agent_query HTTP/1.1" 200 -
192.168.18.1 - - [19/Jun/2026 01:08:25] "POST /agent_query HTTP/1.1" 200 -
192.168.18.1 - - [19/Jun/2026 01:08:42] "POST /agent_query HTTP/1.1" 200 -
```

加载API Key

自己申请的key值

镜像名

页面请求和响应

上图的ZHIPU_API_KEY 是当前代码设置的变量名。可以自己修改熟悉的名称，前提是修改main.py代码27行的字符串内容即可。
如果是非智谱模型，也可以修改，将model = "你自己的模型名称"

```
23
24 # 使用智谱 GLM-4.5 模型
25 model = ChatOpenAI(
26     model="glm-4.5",
27     api_key=os.getenv("ZHIPU_API_KEY"),
28     base_url="https://open.bigmodel.cn/api/paas/v4",
29     callbacks=[langfuse_callback] if langfuse_callback else [],
30 )
31
```

KEY的名称，其它模型

修改此就可以

下图是镜像构建执行过程中产生的日志，用于和自己在镜像构建过程中执行的情况进行比较。

```
补充内容：镜像构建过程日志

1 root@ubuntuserver:/home/ubuntu/bigip# ll
2 total 60
3 drwxr-xr-x 3 root root 4096 Jun 19 00:24 ./
```

```

 4 drwxr-x--- 5 ubuntu ubuntu 4096 Jun 19 00:30 ../
 5 -rw-r--r-- 1 root root 1050 Jun 18 15:49 app.py
 6 -rw-r--r-- 1 root root 4905 Jun 18 15:45 bigip.conf
 7 -rw-r--r-- 1 root root 374 Jun 19 00:24 Dockerfile
 8 -rw-r--r-- 1 root root 2291 Jun 19 00:21 .env.example
 9 -rw-r--r-- 1 root root 7010 Jun 19 00:02 main.py
10 -rw-r--r-- 1 root root 5439 Jun 18 16:07 README.md
11 -rw-r--r-- 1 root root 115 Jun 19 00:21 requirements.txt
12 drwxr-xr-x 2 root root 4096 Jun 18 15:43 templates/
13 -rw-r--r-- 1 root root 7174 Jun 19 00:14 tools.py
14 root@ubuntuserver:/home/ubuntu/bigip# docker build -t agent_bigip:v1 .
15 [+] Building 2.9s (10/10) FINISHED
    docker:default
16 => [internal] load build definition from Dockerfile
    0.0s
17 => => transferring dockerfile: 413B
    0.0s
18 => [internal] load metadata for docker.io/library/python:3.11-slim
    0.2s
19 => [internal] load .dockerignore
    0.0s
20 => => transferring context: 2B
    0.0s
21 => [1/5] FROM docker.io/library/python:3.11-
slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca 0.0s
22 => => resolve docker.io/library/python:3.11-
slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca 0.0s
23 => [internal] load build context
    0.0s
24 => => transferring context: 308B
    0.0s
25 => CACHED [2/5] WORKDIR /app
    0.0s
26 => CACHED [3/5] COPY requirements.txt .
    0.0s
27 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
    0.0s
28 => CACHED [5/5] COPY . .
    0.0s
29 => exporting to image
    2.5s
30 => => exporting layers
    0.0s
31 => => exporting manifest sha256:fe9d893527b6a57d22e33295f5b91a8459978765c80e341b65d0c261c4afcdef
    0.0s
32 => => exporting config sha256:19142c687fd6cdc733cc92f03208707808cbb6a7d60d033497ab60b44bc1dcd5
    0.0s
33 => => exporting attestation manifest
sha256:d1b99f5b72d7bc02c510e08e9cfb2dfc6ba5f99b838c53bbeaf2f2df9e063dfe 0.0s
34 => => exporting manifest list sha256:724108e5f81aa89296e126142dda5a6c8ab4506c56d7cef2fbee2327a3efa655
    0.0s
35 => => naming to docker.io/library/agent_bigip:v1
    0.0s
36 => => unpacking to docker.io/library/agent_bigip:v1
    2.4s
37 root@ubuntuserver:/home/ubuntu/bigip# docker images |grep agent
38 agent_bigip:v1 724108e5f81a 355MB 77.2MB
39 root@ubuntuserver:/home/ubuntu/bigip#

```